



Evaluate the impact of quarterly earnings reports on stock price movement

FRE-GY 6883 Financial Computing

Team 2

Xiaoyi Dong, Kexin Tan, Yiqun Ma, Jinjing Wang, Haolin Liu

2026.05.09

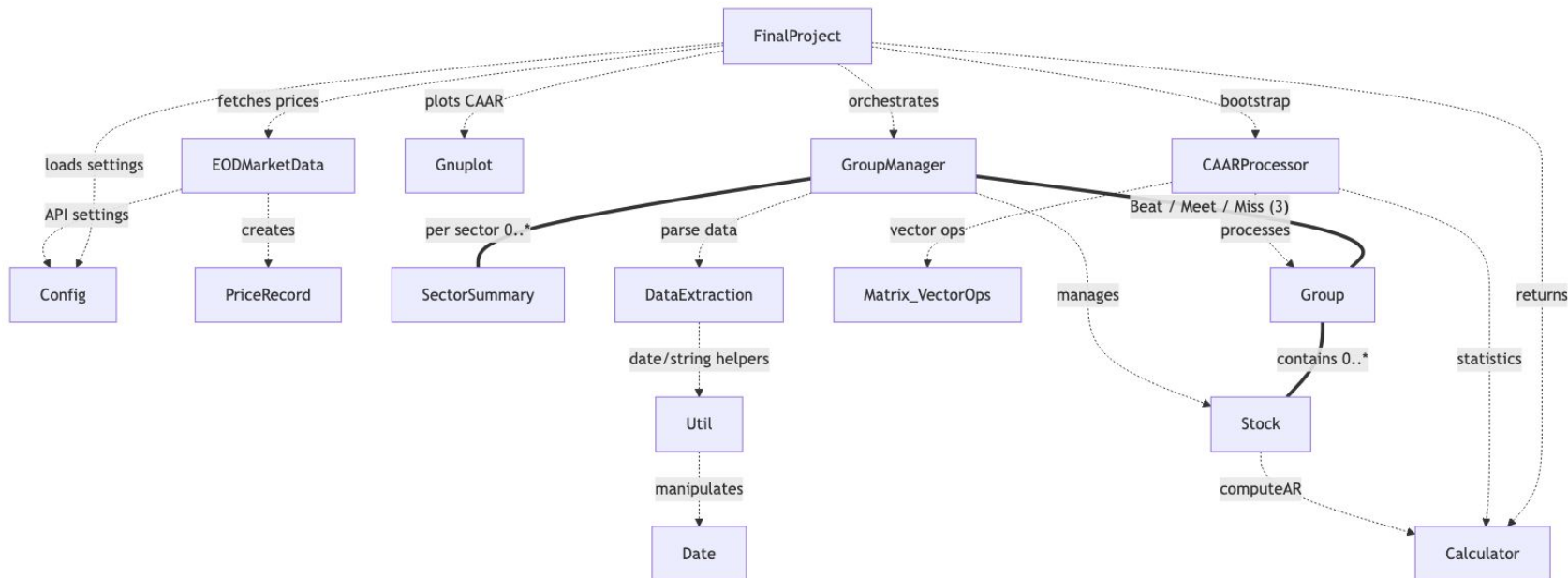
Executive Summary

- **Sector-neutral event study on Russell 3000 earnings announcements**
- **Stocks grouped into Beat / Meet / Miss using earnings surprise %**
- **Top and bottom 2% outliers removed within each sector**
- **Historical prices retrieved using libcurl + EOD Historical Data API**
- **Bootstrap sampling used to compute $E[AAR]$, AAR_STD , $E[CAAR]$, $CAAR_STD$, t-statistics, and confidence intervals**
- **Final outputs include statistical tables, sector summary reports, and CAAR visualization using gnuplot**

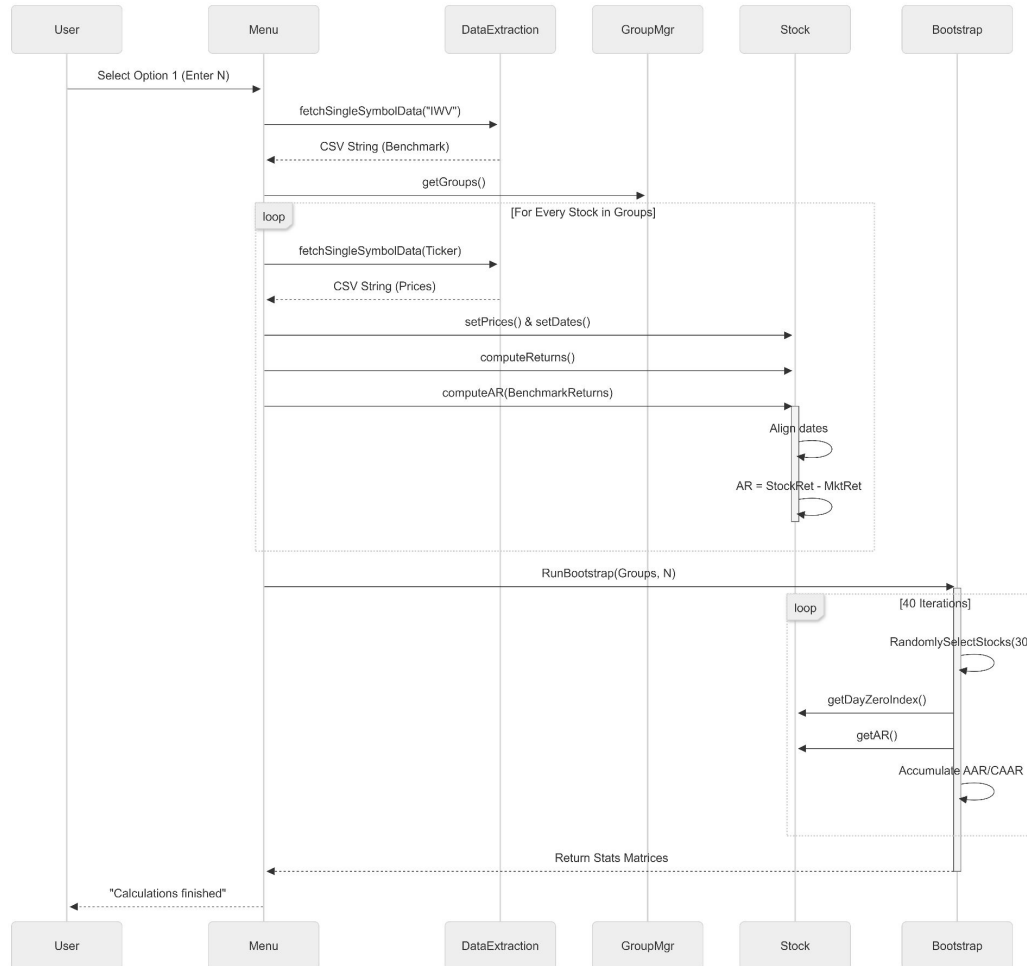
Stock Selection Method

- **Stocks were selected from the Russell 3000 earnings announcement dataset**
- **Only stocks with valid earnings and historical price data were retained**
- **Sector information was matched using ETF holdings data**
- **Stocks were grouped by earnings surprise percentage within each sector**
- **Top and bottom 2% outliers were removed before forming Beat / Meet / Miss groups**

Class Relationship



Data Flow



Data Preparation Layer: From Raw Files/API to Stock Objects

«Utility» Config
public: +int N +int bootstrap_samples +int bootstrap_iterations +string api_token_file +string symbol_file +string earnings_file +string benchmark_symbol +string exchange_suffix +string api_base_url
+load(const string& filename) +isValidN(int value) : bool

Config

- Loads N, file paths, benchmark symbol, API URL
- Validates N between 30 and 60

«Utility» Util	Date
public: +parseDate(const string& s) : Date +formatDate(const Date& d) : string +isLeapYear(int y) : bool +daysInMonth(int y, int m) : int +addDays(const Date& d, int n) : Date +addDays(const string& date, int n) : string +trim(const string& s) : string +toUpper(string s) : string +startsWith(const string& text, const string& prefix):bool +splitCsvLine(const string& line):vector<string>	public: +int year +int month +int day

Util / Date

- Handles date parsing, date shifting, string cleaning, and CSV splitting

DataExtraction
private: -MemoryStruct memory -myrealloc(void* ptr, size_t size) : void* -write_data2(void* ptr, size_t size, size_t nmemb, void* data) : int
public: +parseEarningsData(const string& earningsFile, const string& sectorFile) : map<string, vector<Stock>> +formatDate(const string& date) : string +cleanString(const string& str) : string +validateEarningsData(const string& ticker, double estimate, double reported) : bool +populateSymbolVector(vector<string>& symbols, const string& symbolFile) +fetchMarketDataForSymbols(...) : bool +fetchSingleSymbolData(...) : string +readApiToken(const string& filename) : string +findDayZeroIndex(const string& announceDate, const vector<string>& marketDates) : int

DataExtraction

- Reads earnings CSV and ETF holdings CSV
- Matches ticker symbols with sectors
- Creates Stock objects by sector
- Main container: map<string, vector<Stock>>

Data Preparation Layer: From Raw Files/API to Stock Objects

```
class EODMarketData {
private:
    -string token_
    -string symbolFile_
    -string benchmarkSymbol_
    -string exchangeSuffix_
    -string apiBaseUrl_

    +loadToken(const string& file) : string
    +normalizeSymbol(const string& symbol) : string
    +parseCsvResponse(const string& csvText) :
        vector<PriceRecord>

    +fetchRawUrl(const string& url) : string
    +buildFundamentalsHoldingsUrl() : string

public:
    +EODMarketData(const string& symbolFile,
        const string& tokenFile)

    +getSymbols() : vector<string>
    +getBenchmarkSymbol() : string
    +getApiToken() : string

    +buildRequestUrl(const string& symbol,
        const string& from, const string& to) : string

    +fetchPrices(const string& symbol,
        const string& from, const string& to) :
        vector<PriceRecord>

    +extractWindow(...) : bool
    +fetchPriceWindow(...) : bool
    +fetchAlignedStockAndBenchmarkWindows(...) : bool

    +downloadToFile(...) : bool
    +downloadRussell3000HoldingsCsv(...) : bool
}
```

PriceRecord

```
class PriceRecord {
public:
    +string date

    +double adjClose
}
```

```
class Stock {
private:
    -string symbol
    -string sector
    -string group
    -double estEPS
    -double repEPS
    -double surprise
    -double surprisePct
    -string announceDate
    -string periodEnding
    -vector<double> adjClose
    -vector<string> dates
    -vector<double> returns
    -vector<double> AR

    +Stock(string sym=, string sec=)
    +setEPS(double est, double rep)
    +setGroup(const string& g)
    +setAnnounceDate(const string& date)
    +setPeriodEnding(const string& date)
    +setPrices(const vector<double>& p)
    +setDates(const vector<string>& d)
    +getSymbol() : string
    +getSector() : string
    +getGroup() : string
    +getEstEPS() : double
    +getRepEPS() : double
    +getSurprise() : double
    +getSurprisePct() : double
    +getAnnounceDate() : string
    +getPeriodEnding() : string
    +computeReturns()
    +computeAR(const vector<double>& marketReturns)
    +getDayZeroIndex() : int
    +getPricesAroundDayZero(int daysBefore, int daysAfter) : vector<double>
    +getDatesAroundDayZero(int daysBefore, int daysAfter) : vector<string>
    +getReturnsAroundDayZero(int daysBefore, int daysAfter) : vector<double>
    +getReturns() : const vector<double>&
    +getAR() : const vector<double>&
    +getDates() : const vector<string>&
    +getPrices() : const vector<double>&
}
```

EODMarketData

- Uses libcurl to retrieve adjusted close prices
- Builds API request URLs
- Parses CSV response into PriceRecord vector
- Aligns stock and IWW benchmark windows

PriceRecord

- Stores one daily price record: date + adjusted close

Stock

- Stores earnings profile, prices, dates, returns, and AR
- computeReturns(): calculates log returns
- computeAR(): calculates abnormal returns using IWW benchmark



Grouping and Data Organization Layer

```
Group

-name : string
-stocks : map<string, Stock>
-AAR : vector<double>
-CAAR : vector<double>

+Group(n : string = "")
+addStock(s : const Stock&)
+getStocks() : const map<string, Stock>&
+collectStockPointers(targetList : vector<Stock*>&)
+findStock(ticker : const string&) : Stock*
+getName() : string
```

Group

- Represents one group: Beat, Meet, or Miss
- Stores stocks using map<string, Stock>
- Key = stock ticker
- Provides grouped Stock data to CAARProcessor

```
GroupManager

private:
-map<string, vector<Stock>> sectorStocks
-map<string, Group> groups
-map<string, SectorSummary> sectorSummary

-removeOutliers(const vector<Stock>&
stocks, double pct) : vector<Stock>
-splitIntoGroups(const vector<Stock>&
stocks) : map<string, vector<Stock>>

public:
+GroupManager()

+loadSectorData(const string& earningsFile,
const string& sectorFile)

+buildGroups()

+getGroups() : map<string, Group>&
+getSectorSummary() : const map<string, SectorSummary>&

+getAllStockPointers() : vector<Stock*>

+findStock(const string& ticker) : Stock*

+printSectorSummary()
```

GroupManager

- Performs sector-neutral grouping
- Sorts stocks by surprise percentage within each sector
- Removes top 2% and bottom 2% outliers
- Splits remaining stocks into Beat, Meet, and Miss
- Main containers:
map<string, vector<Stock>> sectorStocks
map<string, Group> groups
map<string, SectorSummary> sectorSummary

```
SectorSummary

public:
+int total
+int retained
+int excluded
+int beat
+int meet
+int miss

+SectorSummary()
```

SectorSummary

- Stores sector-level counts:
total, retained, excluded, Beat, Meet, Miss
- Used for Option 5 sector summary report

Calculation, Bootstrap, and Output Layer

«Utility» Calculator	«Utility» MatrixUtils	Gnuplot	CAARProcessor
<pre>public: vector<double> double double vector<double> pair~double, double~ +logReturn(const vector<double>& prices): +abnormalReturn(const vector<double>& stockReturns, const vector<double>& marketReturn) : vector<double> +mean(const vector<double>& v) : +stddev(const vector<double>& v) : +cumulativeSum(const vector<double>& v): +tStat(double meanValue, double stdValue) : double +confidenceInterval(double meanValue, double stdValue, double confidenceLevel) : +printStatisticsTable(...)</pre>	<pre>public: +typedef vector<double> Vector +typedef vector<Vector> Matrix +operator*(const Matrix& C, const Vector& V) : Vector +operator*(const double& a, const Vector& V) : Vector +operator*(const Vector& V, const Vector& W) : Vector +operator+(const double& a, const Vector& V) : Vector +operator+(const Vector& V, const Vector& W) : Vector +operator-(const Vector& V, const double& a) : Vector +operator-(const Vector& V, const Vector& W) : Vector +operator/(const Vector& V, const Vector& W) : Vector +operator/(const Vector& V, const double a) : Vector +operator^(const Vector& V, const Vector& W) : double +exp(const Vector& V) : Vector</pre>	<pre>private: -string title -string xlabel -string ylabel -vector<double> xData -vector<vector<double>> yData -vector<string> seriesNames -FILE* gnuplotPipe public: +Gnuplot(...) +Gnuplot(..., const vector<string>& seriesNames_) +~Gnuplot() +plot() : bool</pre>	<pre>-Vector E_AAR -Vector AAR_STD -Vector E_CAAR -Vector CAAR_STD -int N -int samples -int iterations +CAARProcessor(int n_days=30, int sample_size=30, int iter=40) +process(Group& group) : void +get_E_AAR() : const Vector& +get_AAR_STD() : const Vector& +get_E_CAAR() : const Vector& +get_CAAR_STD() : const Vector& +get_N() : int +printSignificanceTable(bool isCAAR=false) : void +consolidateGroupMatrices(map<string, CAARProcessor>&, vector<string>&) : Matrix +printGroupPerDayResults(Matrix&, vector<string>&, int, int) : void</pre>

Calculator

- Provides all math/statistical functions
- logReturn, abnormalReturn, mean, stddev
- cumulativeSum, tStat, confidenceInterval

Matrix

- Defines Vector = vector<double>
- Defines Matrix = vector<Vector>
- Overloads vector/matrix arithmetic operator

Gnuplot

- Uses E[CAAR] values from CAARProcessor
- Plots Beat / Meet / Miss expected CAAR curves

CAARProcessor

- Core bootstrap engine
- Receives one Group as input
- Samples 30 stocks with replacement
- Repeats 40 iterations
- Computes E[AAR], AAR_STD, E[CAAR], CAAR_STD
- Generates AAR and CAAR statistical tables

Program Output

FRE 6883 Final Project:
Current N: 60
1. Enter N and retrieve historical price data
2. Pull information for one stock
3. Show AAR/CAAR statistics for one group
4. Plot expected CAAR for all groups with gnuplot
5. Show sector summary report
6. Exit
Choose an option: 2
Enter ticker: AAPL

Ticker: AAPL
Group: Meet
Sector: Information Technology
Earnings announcement date: 2025-10-30
Period ending: 2025-09-30
Estimated EPS: 1.760000
Reported EPS: 1.850000
Surprise: 0.090000
Surprise %: 5.113636
Daily adjusted closing prices

Day(t)	Date	Adj Close
-60	2025-08-06	212.603000
-59	2025-08-07	219.362500
-58	2025-08-08	228.654200
-57	2025-08-11	226.747800

Cumulative daily returns

Day(t)	Date	Return	Cumulative
-59	2025-08-07	0.031299	0.031299
-58	2025-08-08	0.041485	0.072784
-57	2025-08-11	-0.008372	0.064412
-56	2025-08-12	0.010814	0.075226
-55	2025-08-13	0.015897	0.091123
-54	2025-08-14	-0.002360	0.088763
-53	2025-08-15	-0.005126	0.083638
-52	2025-08-18	-0.003027	0.080611
-51	2025-08-19	-0.001430	0.079180
-50	2025-08-20	-0.019932	0.059248

Choose an option: 3
Enter group (Beat, Meet, or Miss): Beat

AAR Statistical Significance Table (95% CI, 40 bootstrap iterations, N=60)

Day(t)	Mean	STD	t-stat	CI Lower	CI Upper
-59	-0.000264	0.006811	-0.038715	-0.013613	0.013086
-58	0.002018	0.005145	0.392150	-0.008067	0.012103
-57	-0.000478	0.004470	-0.106877	-0.009238	0.008283

CAAR Statistical Significance Table (95% CI, 40 bootstrap iterations, N=60)

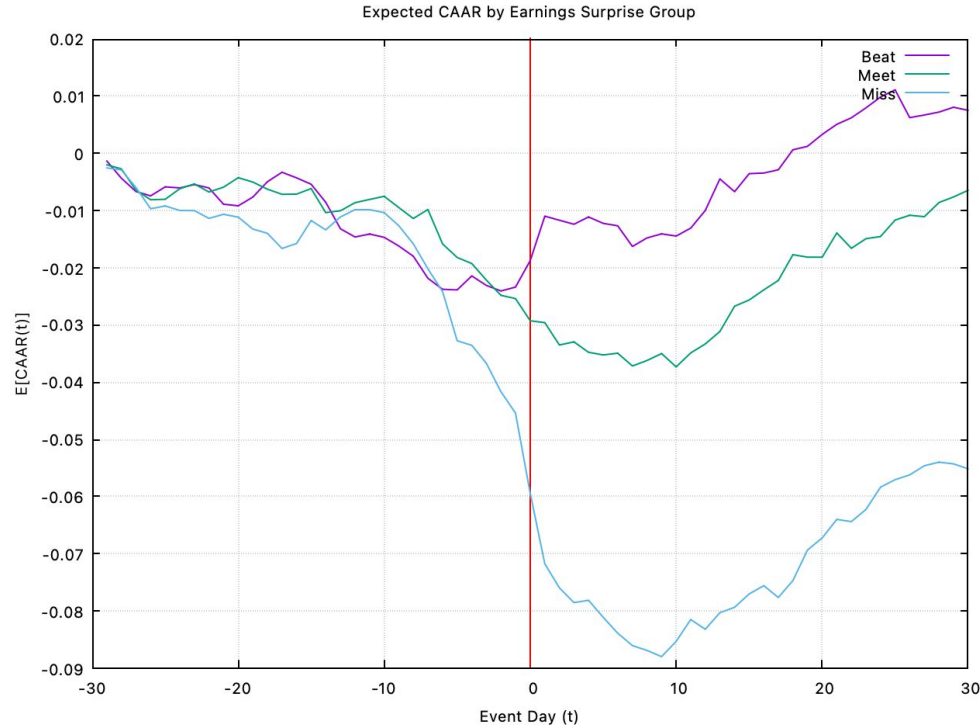
Day(t)	Mean	STD	t-stat	CI Lower	CI Upper
-59	-0.000264	0.006811	-0.038715	-0.013613	0.013086
-58	0.001754	0.009710	0.180642	-0.017278	0.020786
-57	0.001276	0.009222	0.138412	-0.016798	0.019351

Choose an option: 5

Sector Summary Report

Sector	Total	Retained	Excluded	Beat	Meet	Miss
Communication	106	102	4	34	34	34
Consumer Discretionary	280	270	10	90	90	90
Consumer Staples	100	96	4	32	32	32
Energy	113	109	4	36	37	36
Financials	441	425	16	142	141	142
Health Care	392	378	14	126	126	126
Industrials	368	354	14	118	118	118
Information Technology	304	292	12	97	98	97
Materials	109	105	4	35	35	35
Other	1	1	0	0	1	0
Real Estate	138	134	4	45	44	45
Unknown	89	87	2	29	29	29
Utilities	68	66	2	22	22	22
ALL	2509	2419	90	806	807	806

Program Output (Post-Earnings CAAR Performance by Earnings Surprise Group)



Conclusion

- Earnings announcements create clear post-event stock price reactions.
- Beat stocks show positive post-announcement momentum.
- Miss stocks experience the strongest negative abnormal returns.
- Meet stocks show relatively limited market reactions.
- The CAAR trends clearly separate after the earnings announcement date.
- Our results suggest that earnings surprise is an important driver of short-term stock price movement.

Enhancements / Additional Features

- Added parallel API calls with `std::async`
- Fetches up to 20 stocks at once
- Speeds up historical price retrieval
- Skips failed/incomplete data safely
- Shows progress for Beat, Meet, and Miss groups

Reflections

- Increase the number of bootstrap iterations for more stable estimates.
- Expand the event window to test longer-term price reactions.

References

- FRE-GY 6883 Financial Computing Course Slides and Project Materials

Thank you!

❖ Task Allocation

- . Xiaoyi Dong: EODMarketData.h, EODMarketData.cpp, Config.h, Config.cpp, Util.h, Util.cpp
- . Kexin Tan: Calculator.h, Calculator.cpp, Matrix.h, Matrix.cpp, Stock.h, Stock.cpp
- . Yiqun Ma: Group.h, Group.cpp, GroupManager.h, GroupManager.cpp
- . Jinjing Wang: CAARProcessor.h, CAARProcessor.cpp
- . Haolin Liu: FinalProject.cpp, Gnuplot.h, Gnuplot.cpp, Makefile, README.md